

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

LABORATORY WORK NO. 1

**Classical Ciphers: Caesar Cipher and Caesar
Cipher with Permutation for the Romanian
Alphabet**

Course: Cryptography and Information Security

Author:

Vitalie VASILEAN

Student of group FAF-231

Verified by:

Department of ISA

Lecturer / Supervisor

Contents

1	Introduction and Laboratory Objectives	2
2	Short Theory	2
2.1	The Classical Caesar Cipher	2
2.1.1	Encryption and Decryption Formulas	3
2.1.2	Key Space and Security of Classic Caesar	3
2.2	The Caesar Cipher with a Permutation (Two-Key Caesar)	4
2.2.1	Permutation Construction Algorithm	4
2.2.2	Encryption and Decryption with Permutation	4
2.2.3	Key Space and Security Analysis of the Permuted Cipher	5
3	Implementation	5
3.1	Architecture and Technologies	5
3.2	Key Implementation Fragments	6
3.2.1	Alphabet Definition and Encoding	6
3.2.2	Diacritic and Cedilla Normalization	6
3.2.3	Permuted Alphabet Construction	6
3.2.4	Encryption and Decryption Logic	7
3.2.5	Defensive Input Validation and Error Recovery	8
4	Results	10
4.1	Task 1.1: Classic Caesar Cipher Execution	10
4.1.1	Test 1.1.1: Encryption and Decryption of Romanian Plaintext	10
4.1.2	Test 1.1.2: Error Handling for Invalid Key and Characters	11
4.2	Task 1.2: Caesar Cipher with Permutation Execution	12
4.2.1	Test 1.2.1: Permuted Alphabet Display, Encryption, and Decryption	12
4.2.2	Manual Verification of Permutation Encryption	13
4.2.3	Test 1.2.2: Error Handling for Keyword Validation	13
5	Conclusions	14
6	Source Code	15
6.1	Building and Running the Project	15

1 Introduction and Laboratory Objectives

Classical cryptography represents the foundational basis upon which modern cryptosystems and mathematical cryptanalysis have been developed. Early symmetric encryption algorithms, predominantly substitution and transposition ciphers, were designed to secure sensitive communications through manual transformations of plaintext into unintelligible ciphertext.

The primary objectives of this laboratory work are:

1. **Understand the operation of the classical Caesar cipher** through rigorous mathematical formulation of modular arithmetic over finite alphabetic rings.
2. **Analyze the cryptographic weakness of small key spaces**, demonstrating why the standard Caesar cipher is vulnerable to exhaustive brute-force attacks and statistical frequency cryptanalysis.
3. **Implement the Caesar cipher over the extended Romanian alphabet** ($n = 31$ characters), strictly adhering to a predefined letter-to-index mapping without relying on ASCII/Unicode numeric offsets.
4. **Enhance cipher security using a keyword-based alphabet permutation**, observing how the key space dramatically increases from a small integer range to factorial proportions ($31!$).
5. **Implement robust input sanitization, diacritic normalization, and validation recovery**, handling keyboard layout variations (e.g., cedilla vs. comma-below variants) and providing explicit feedback upon invalid user inputs.

2 Short Theory

2.1 The Classical Caesar Cipher

The Caesar cipher, named after Julius Caesar who used it to communicate with his generals, is one of the earliest known monoalphabetic substitution ciphers. Each letter in the plaintext message is shifted by a fixed number of positions k along the ordered alphabet.

Let the alphabet Σ be an ordered set of n distinct symbols:

$$\Sigma = (s_0, s_1, s_2, \dots, s_{n-1}) \quad (1)$$

A bijective encoding function $f : \Sigma \rightarrow \mathbb{Z}_n$ assigns each letter a numeric code corresponding to its 0-indexed position:

$$f(s_i) = i, \quad \text{where } i \in \{0, 1, \dots, n-1\} \quad (2)$$

For the Romanian alphabet, $n = 31$. The assigned numeric codes according to the lab guide specification are presented in Table 1.

Table 1: Letter encoding of the Romanian alphabet ($n = 31$)

Letter	Code	Letter	Code	Letter	Code	Letter	Code
A	0	F	7	K	13	P	18
Ă	1	G	8	L	14	Q	19
Â	2	H	9	M	15	R	20
B	3	I	10	N	16	S	21
C	4	Î	11	O	17	Ș	22
D	5	J	12			T	23
E	6					Ț	24
U = 25		V = 26		W = 27		X = 28	
Y = 29				Z = 30			

2.1.1 Encryption and Decryption Formulas

Let $x = f(m)$ represent the numeric code of a plaintext character m , and $y = f(c)$ represent the code of the corresponding ciphertext character c . The secret key is the shift parameter:

$$k \in \{1, 2, 3, \dots, n-1\} \quad (3)$$

The shift $k = 0$ is excluded from the valid key space because it represents the identity transformation $e_0(x) = x$, leaving the plaintext unchanged.

The encryption function $e_k(x)$ and decryption function $d_k(y)$ are defined by modular arithmetic:

$$c = e_k(x) = (x + k) \pmod{n} \quad (4)$$

$$m = d_k(y) = (y - k) \pmod{n} \quad (5)$$

In modular arithmetic, when the difference $(y - k)$ yields a negative value, the mathematical congruence modulo n requires adding n to wrap back into the non-negative interval $[0, n-1]$:

$$(y - k) \pmod{n} \equiv (y - k + n) \pmod{n} \quad (6)$$

This explicit adjustment is essential in programming languages such as C#, where the native remainder operator `%` returns a negative remainder when the dividend is negative.

2.1.2 Key Space and Security of Classic Caesar

The key space $\mathcal{K}$ for the classic Caesar cipher over an alphabet of length n consists of:

$$|\mathcal{K}| = n - 1 \quad (7)$$

For the Romanian alphabet ($n = 31$), the key space is strictly:

$$|\mathcal{K}| = 31 - 1 = 30 \quad (8)$$

Because $|\mathcal{K}| = 30$ is minuscule, the cipher provides no meaningful security against an adversary:

- **Exhaustive Search (Brute-Force):** An attacker can decrypt the intercepted ciphertext using all 30 potential values of k . Since natural Romanian language exhibits clear lexical structure, exactly one candidate key yields intelligible plaintext. An exhaustive search takes a fraction of a millisecond on any computing device.

- **Frequency Analysis:** In the Romanian language, character distribution is non-uniform (letters such as *E, I, A, R, T* occur with high frequency, while *Q, W, Y, K* are rare). Because Caesar shifting preserves the shape of the letter frequency distribution curve merely shifted by k slots, comparing the most frequent ciphertext letter directly identifies the key k .

2.2 The Caesar Cipher with a Permutation (Two-Key Caesar)

To mitigate the vulnerability caused by the minuscule key space of the standard Caesar cipher, the natural order of the alphabet is permuted before applying the shift. This variant employs a two-part key (k_1, k_2) :

- **Key 1** (k_1): The numeric cyclic shift, where $k_1 \in \{1, 2, \dots, n - 1\}$.
- **Key 2** (k_2): A secret keyword consisting of letters from the Romanian alphabet with a minimum length of 7 characters.

2.2.1 Permutation Construction Algorithm

The permuted alphabet Σ_{perm} is generated systematically:

1. Normalize k_2 by converting to uppercase, stripping spaces, and mapping cedilla characters to standard Romanian comma-below letters.
2. Extract the distinct characters of k_2 in their order of first appearance, dropping any duplicate occurrences.
3. Append the remaining letters of the original Romanian alphabet Σ that were not present in k_2 , preserving their standard alphabetical order.

For example, if the keyword is $k_2 = \text{“CRIPTOGRAFIE”}$ over the 31-letter Romanian alphabet:

- Distinct letters from keyword: **C, R, I, P, T, O, G, A, F, E** (10 unique letters; subsequent occurrences of R, I, and E are discarded).
- Remaining Romanian letters: **Ă, Â, B, D, H, Î, J, K, L, M, N, Q, S, Ș, Ț, U, V, W, X, Y, Z** (21 letters).
- Resulting permuted alphabet Σ_{perm} of length 31:

$$\Sigma_{\text{perm}} = \text{C R I P T O G A F E } \check{\text{A}} \hat{\text{A}} \text{ B D H } \hat{\text{I}} \text{ J K L M N Q S } \text{Ș } \text{Ț U V W X Y Z}$$

2.2.2 Encryption and Decryption with Permutation

The numeric code of a character is defined as its 0-indexed position within the **permuted alphabet** Σ_{perm} . Let $\text{index}_{\Sigma_{\text{perm}}}(ch)$ denote the index of character ch in Σ_{perm} , and let $\Sigma_{\text{perm}}[j]$ denote the character at index j .

Encryption: For each plaintext letter m_i :

$$x_i = \text{index}_{\Sigma_{\text{perm}}}(m_i) \tag{9}$$

$$c_i = \Sigma_{\text{perm}}[(x_i + k_1) \pmod{n}] \tag{10}$$

Decryption: For each ciphertext letter c_i :

$$y_i = \text{index}_{\Sigma_{\text{perm}}}(c_i) \tag{11}$$

$$m_i = \Sigma_{\text{perm}}[(y_i - k_1 + n) \pmod{n}] \tag{12}$$

2.2.3 Key Space and Security Analysis of the Permuted Cipher

The number of possible orderings (permutations) of an alphabet of size $n = 31$ is given by factorial $n!$:

$$31! = 8,222,838,654,177,922,817,725,562,880,000,000 \approx 8.2228 \times 10^{33} \approx 2^{112.6} \quad (13)$$

Accounting for the numeric shift $k_1 \in \{1, \dots, 30\}$, the theoretical number of key pairs (k_1, k_2) is:

$$(n - 1) \times n! = 30 \times 31! \approx 2.4668 \times 10^{35} \quad (14)$$

However, because applying a cyclic shift k_1 to an already permuted alphabet is mathematically equivalent to another valid permutation of the alphabet, the total number of distinct bijective substitution mappings cannot exceed $n! = 31!$.

Cryptanalytic Implications:

- **Immunity to Brute-Force:** A key space of order $31! > 2^{112}$ renders exhaustive key search computationally impossible. Even a supercomputer testing a trillion (10^{12}) keys per second would require more than 2.6×10^{14} years to exhaust the space.
- **Vulnerability to Frequency Analysis:** Despite the massive key space, the cipher remains a **monoalphabetic substitution cipher**. Each plaintext symbol consistently maps to one and only one ciphertext symbol. Consequently, letter frequency distributions, digraph frequencies (e.g., *DE*, *ÎN*, *RE*, *LA*), and trigraphs in the Romanian language remain preserved in the ciphertext. A cryptanalyst can break the cipher using frequency analysis and pattern matching without needing to search the key space.

3 Implementation

3.1 Architecture and Technologies

The solution is developed in **C#** using the **.NET 10.0** runtime. The codebase follows object-oriented and modular programming principles, structured into four cohesive components:

- **Program.cs:** The application entry point. Sets the standard console input and output encodings to **UTF-8** to properly render Romanian diacritics, and initiates the user interaction loop.
- **UserMenu.cs:** Implements the console user interface, menus, and defensive input processing routines. Reads and validates messages, keys, and keywords, reprompting whenever invalid input is encountered.
- **CaesarCipher.cs:** Encapsulates the 1-key classic Caesar cipher routines: key validation ($1 \leq k \leq 30$), text normalization, and modular arithmetic.
- **CaesarCipher2Key.cs:** Encapsulates the 2-key permuted Caesar cipher routines: key 1 validation, key 2 keyword validation (minimum 7 characters, valid alphabet), permutation builder, and encryption/decryption routines.

3.2 Key Implementation Fragments

3.2.1 Alphabet Definition and Encoding

To strictly adhere to Requirement 2 of Task 1.1, the cipher does not compute shifts using underlying ASCII or Unicode scalar values. Instead, a custom Romanian alphabet string represents the exact ordered sequence of Table 1:

```
1 string romanianAlphabet = "AĂÂBCDEFGHIÎJKLMNOPSȘȚUVWXYZ";
```

Listing 1: Romanian Alphabet Definition

The numeric code of each letter is determined via `alphabet.IndexOf(c)`, mapping character `c` directly to its index in `[0, 30]`.

3.2.2 Diacritic and Cedilla Normalization

Romanian keyboards frequently alternate between the official comma-below characters (Ș, Ț) and legacy cedilla-accented characters (Ş, Ţ). To ensure seamless compatibility regardless of the user's keyboard layout, an input normalization routine transforms cedilla characters into their standard comma-below counterparts:

```
1 public static string Normalize(string message)
2 {
3     return message
4         .Replace('ş', 'ș')
5         .Replace('Ş', 'Ș')
6         .Replace('ţ', 'ț')
7         .Replace('Ț', 'Ț');
8 }
```

Listing 2: Character Normalization Method in CaesarCipher.cs

Comment: This method normalizes all incoming plaintext, ciphertext, and keyword strings before any cryptographic processing, preventing rejection of valid Romanian text typed with legacy keyboard drivers.

3.2.3 Permuted Alphabet Construction

The permutation of the alphabet is constructed using a `HashSet<char>` to track unique characters, and a `StringBuilder` to assemble the new sequence:

```
1 public static string BuildPermutedAlphabet(string key2, string
2     alphabet)
3 {
4     string normalizedKey = Normalize(key2).Replace(" ",
5     "").ToUpper();
6     StringBuilder result = new();
7     HashSet<char> seen = [];
8
9     // Step 1: Add unique letters from keyword in
10    first-occurrence order
11    foreach (char c in normalizedKey)
12    {
13        if (!seen.Contains(c))
```

```

11     {
12         seen.Add(c);
13         result.Append(c);
14     }
15 }
16
17 // Step 2: Append remaining alphabet letters in natural
18 // alphabetical order
19 foreach (char c in alphabet)
20 {
21     if (!seen.Contains(c))
22     {
23         seen.Add(c);
24         result.Append(c);
25     }
26 }
27 return result.ToString();
28 }

```

Listing 3: Alphabet Permutation Construction in CaesarCipher2Key.cs

Comment: The `HashSet<char>` provides $O(1)$ lookup to ensure each letter appears exactly once. The resulting string has length 31 and begins with the keyword's distinct letters, followed by the unused letters in their original order.

3.2.4 Encryption and Decryption Logic

The core encryption and decryption methods operate on the permuted alphabet for the 2-key variant (and on the standard alphabet for the 1-key variant). Spaces are stripped, characters are uppercased, and modular shifts are applied:

```

1 public static string Encrypt(string normalMessage, int key,
2   string key2, string alphabet)
3 {
4     Key1Validator(key, alphabet);
5     Key2Validator(key2, alphabet);
6     normalMessage = Normalize(normalMessage);
7     StringValidator(normalMessage, alphabet);
8
9     string permutedAlphabet = BuildPermutedAlphabet(key2,
10    alphabet);
11
12    StringBuilder encodedMessage = new();
13    for (int i = 0; i < normalMessage.Length; i++)
14    {
15        if (!char.IsWhiteSpace(normalMessage[i]))
16        {
17            int index =
18                permutedAlphabet.IndexOf(char.ToUpper(normalMessage[i]));
19            int shiftedIndex = (index + key) %
20                permutedAlphabet.Length;
21            char new_c = permutedAlphabet[shiftedIndex];

```

```

18         encodedMessage.Append(new_c);
19     }
20 }
21 return encodedMessage.ToString();
22 }
23
24 public static string Decrypt(string normalMessage, int key,
25     string key2, string alphabet)
26 {
27     Key1Validator(key, alphabet);
28     Key2Validator(key2, alphabet);
29     normalMessage = Normalize(normalMessage);
30     StringValidator(normalMessage, alphabet);
31
32     string permutedAlphabet = BuildPermutedAlphabet(key2,
33     alphabet);
34
35     StringBuilder decodedMessage = new();
36     for (int i = 0; i < normalMessage.Length; i++)
37     {
38         if (!char.IsWhiteSpace(normalMessage[i]))
39         {
40             int index =
41             permutedAlphabet.IndexOf(char.ToUpper(normalMessage[i]));
42             // Handle negative remainders in modular arithmetic
43             int shiftedIndex = (index - (key %
44             permutedAlphabet.Length) + permutedAlphabet.Length)
45             % permutedAlphabet.Length;
46             char new_c = permutedAlphabet[shiftedIndex];
47             decodedMessage.Append(new_c);
48         }
49     }
50     return decodedMessage.ToString();
51 }

```

Listing 4: Encryption and Decryption in CaesarCipher2Key.cs

Comment: In decryption, the expression $(\text{index} - (\text{key} \% n) + n) \% n$ guarantees a non-negative dividend before computing the remainder, correctly wrapping negative differences back into $[0, n - 1]$.

3.2.5 Defensive Input Validation and Error Recovery

Per Requirement 5 of Task 1.1 and Requirement 5 of Task 1.2, any invalid input must produce a clear error message stating the allowed range or rejected character, and prompt the user again without crashing:

```

1 private int? ReadValidKey(string prompt)
2 {
3     while (true)
4     {
5         Console.Write(prompt);
6         string? raw = Console.ReadLine();

```

```

7         if (raw == null) return null;
8         raw = raw.Trim().Trim('\uFEFF');
9         if (int.TryParse(raw, out int key) && key >= 1 && key <=
alphabet.Length - 1)
10            {
11                return key;
12            }
13            Console.WriteLine($"Error: Invalid key. Allowed range is
an integer between 1 and {alphabet.Length - 1} inclusive.");
14            Console.WriteLine("Please enter the key again.");
15        }
16    }
17
18    private string? ReadValidKey2(string prompt)
19    {
20        while (true)
21        {
22            Console.Write(prompt);
23            string? raw = Console.ReadLine();
24            if (raw == null) return null;
25            string key2 = raw;
26            string normalized =
CaesarCipher2Key.Normalize(key2).Replace(" ", "");
27            if (normalized.Length < 7)
28            {
29                Console.WriteLine($"Error: Key 2 (keyword) must be
at least 7 characters long. (Entered length:
{normalized.Length}).");
30                Console.WriteLine("Please enter the keyword again.");
31                continue;
32            }
33
34            bool valid = true;
35            foreach (char c in normalized)
36            {
37                if (!alphabet.Contains(char.ToUpper(c)))
38                {
39                    Console.WriteLine($"Error: Invalid character
'{c}' rejected in keyword. Only Romanian alphabet letters are
allowed.");
40                    valid = false;
41                    break;
42                }
43            }
44            if (valid) return key2;
45            Console.WriteLine("Please enter the keyword again.");
46        }
47    }

```

Listing 5: Interactive Validation and Prompting in UserMenu.cs

Comment: The validation routines loop indefinitely until the user supplies compliant

data. If an illegal character is supplied (e.g., a number or punctuation mark), the specific rejected character is printed to guide the user.

4 Results

This section presents execution outputs verifying all functional requirements, including standard encryption, decryption, alphabet permutation verification, and defensive input validation.

4.1 Task 1.1: Classic Caesar Cipher Execution

4.1.1 Test 1.1.1: Encryption and Decryption of Romanian Plaintext

We test the encryption of the phrase “*cifrul cezar*” with key $k = 3$.

- Plaintext: `cifrul cezar`
- Stripped/Uppercased: `CIFRULCEZAR`
- Mapping trace:
 - C (code 4) $\rightarrow (4 + 3) \bmod 31 = 7 \rightarrow \mathbf{F}$
 - I (code 10) $\rightarrow (10 + 3) \bmod 31 = 13 \rightarrow \mathbf{K}$
 - F (code 7) $\rightarrow (7 + 3) \bmod 31 = 10 \rightarrow \mathbf{I}$
 - R (code 20) $\rightarrow (20 + 3) \bmod 31 = 23 \rightarrow \mathbf{T}$
 - U (code 25) $\rightarrow (25 + 3) \bmod 31 = 28 \rightarrow \mathbf{X}$
 - L (code 14) $\rightarrow (14 + 3) \bmod 31 = 17 \rightarrow \mathbf{O}$
 - C (code 4) $\rightarrow (4 + 3) \bmod 31 = 7 \rightarrow \mathbf{F}$
 - E (code 6) $\rightarrow (6 + 3) \bmod 31 = 9 \rightarrow \mathbf{H}$
 - Z (code 30) $\rightarrow (30 + 3) \bmod 31 = 33 \bmod 31 = 2 \rightarrow \mathbf{\hat{A}}$
 - A (code 0) $\rightarrow (0 + 3) \bmod 31 = 3 \rightarrow \mathbf{B}$
 - R (code 20) $\rightarrow (20 + 3) \bmod 31 = 23 \rightarrow \mathbf{T}$
- Expected Ciphertext: `FKITXOFHÂBT`

```
Console Terminal Session
1 Choose an option:
2 1. 1-Key Caesar Cipher
3 2. 2-Key Caesar Cipher
4 3. Exit
5 > 1
6
7 Choose an option:
8 1. Encrypt
9 2. Decrypt
10 3. Exit
11 > 1
12 Enter the message: cifrul cezar
13 Enter the encryption key (1 - 30): 3
14 Encrypted message: FKITXOFHÂBT
15
16 Choose an option:
17 1. Encrypt
18 2. Decrypt
19 3. Exit
20 > 2
21 Enter the encrypted message: FKITXOFHÂBT
22 Enter the decryption key (1 - 30): 3
23 Decrypted message: CIFRULCEZAR
```

The decrypted message matches the original plaintext CIFRULCEZAR, demonstrating mathematical correctness and complete invertibility over the 31-letter Romanian alphabet.

4.1.2 Test 1.1.2: Error Handling for Invalid Key and Characters

Testing key boundaries (key = 0, key = 35, non-integer) and invalid characters in message text:

```

1 Choose an option:
2 1. 1-Key Caesar Cipher
3 2. 2-Key Caesar Cipher
4 3. Exit
5 > 1
6
7 Choose an option:
8 1. Encrypt
9 2. Decrypt
10 3. Exit
11 > 1
12 Enter the message: salut 123!
13 Error: Invalid character '1' rejected. Allowed characters
    are Romanian alphabet letters (A-Z, Ă, Â, Î, Ș, Ț).
14 Please enter the text again.
15 Enter the message: salut
16 Enter the encryption key (1 - 30): 0
17 Error: Invalid key. Allowed range is an integer between 1
    and 30 inclusive.
18 Please enter the key again.
19 Enter the encryption key (1 - 30): 35
20 Error: Invalid key. Allowed range is an integer between 1
    and 30 inclusive.
21 Please enter the key again.
22 Enter the encryption key (1 - 30): abc
23 Error: Invalid key. Allowed range is an integer between 1
    and 30 inclusive.
24 Please enter the key again.
25 Enter the encryption key (1 - 30): 5
26 Encrypted message: VDQZX
```

The application cleanly rejects the out-of-range keys (0, 35) and non-numeric inputs, explicitly displays the rejected character '1', and repeatedly prompts until valid input is supplied.

4.2 Task 1.2: Caesar Cipher with Permutation Execution

4.2.1 Test 1.2.1: Permuted Alphabet Display, Encryption, and Decryption

We test the two-key cipher with:

- Plaintext: “*securitatea rețelelor*”
- Key 1 (Shift): $k_1 = 7$
- Key 2 (Keyword): $k_2 = \text{“criptografie”}$ (length $12 \geq 7$)

```

1 Choose an option:
2 1. 1-Key Caesar Cipher
3 2. 2-Key Caesar Cipher
4 3. Exit
5 > 2
6
7 Choose an option:
8 1. Encrypt
9 2. Decrypt
10 3. Exit
11 > 1
12 Enter the message: securitatea retelelor
13 Enter key-1 (shift: 1 - 30): 7
14 Enter encryption key-2 (keyword >= 7 characters):
    criptografie
15 Permuted Alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSSŢUVWXYZ
16 Encrypted message: YJARFEÂHÂJHFJÂJUJUBF
17
18 Choose an option:
19 1. Encrypt
20 2. Decrypt
21 3. Exit
22 > 2
23 Enter the encrypted message: YJARFEÂHÂJHFJÂJUJUBF
24 Enter decryption key-1 (shift: 1 - 30): 7
25 Enter decryption key-2 (keyword >= 7 characters):
    criptografie
26 Permuted Alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSSŢUVWXYZ
27 Decrypted message: SECURITATEARETELELOR

```

4.2.2 Manual Verification of Permutation Encryption

Per Requirement 4 of Task 1.2, the permuted alphabet is printed to the console so that the result can be verified by hand:

$$\Sigma_{\text{perm}} = \text{CRIPTOGAFEĂÂBDHÎJKLMNQSSŢUVWXYZ} \quad (15)$$

Table 2 illustrates the manual index lookup for the first few letters of **SECURITATEA...**:

The resulting string **YJARFEÂHÂJH...** perfectly matches the program output, verifying mathematical and algorithmic consistency.

4.2.3 Test 1.2.2: Error Handling for Keyword Validation

Testing invalid keyword scenarios, including keywords shorter than 7 characters and keywords containing non-alphabet characters:

Table 2: Manual verification for message “SECURITATEA” with $k_1 = 7$ under Σ_{perm}

Plaintext (m)	Index in Σ_{perm} (x)	Shifted $(x + 7) \bmod 31$	Ciphertext (c)
S	22	$(22 + 7) \bmod 31 = 29$	Y
E	9	$(9 + 7) \bmod 31 = 16$	J
C	0	$(0 + 7) \bmod 31 = 7$	A
U	25	$(25 + 7) \bmod 31 = 1$	R
R	1	$(1 + 7) \bmod 31 = 8$	F
I	2	$(2 + 7) \bmod 31 = 9$	E
T	4	$(4 + 7) \bmod 31 = 11$	Â
A	7	$(7 + 7) \bmod 31 = 14$	H
T	4	$(4 + 7) \bmod 31 = 11$	Â
E	9	$(9 + 7) \bmod 31 = 16$	J
A	7	$(7 + 7) \bmod 31 = 14$	H

```

1 Choose an option:
2 1. 1-Key Caesar Cipher
3 2. 2-Key Caesar Cipher
4 3. Exit
5 > 2
6
7 Choose an option:
8 1. Encrypt
9 2. Decrypt
10 3. Exit
11 > 1
12 Enter the message: salut
13 Enter key-1 (shift: 1 - 30): 5
14 Enter encryption key-2 (keyword >= 7 characters): secret
15 Error: Key 2 (keyword) must be at least 7 characters long.
    (Entered length: 6).
16 Please enter the keyword again.
17 Enter encryption key-2 (keyword >= 7 characters): parola123
18 Error: Invalid character '1' rejected in keyword. Only
    Romanian alphabet letters are allowed.
19 Please enter the keyword again.
20 Enter encryption key-2 (keyword >= 7 characters): securitate
21 Permuted Alphabet: SECURITAĂÂBDFGHÎJKLMNOPSȚVWXYZ
22 Encrypted message: IFQĂD

```

The system accurately rejects the 6-character keyword "secret", flags the illegal digit in "parola123", and accepts the compliant keyword "securitate".

5 Conclusions

Through this laboratory work, we investigated classical symmetric encryption mechanisms by implementing and analyzing both the standard Caesar cipher and the keyword-permuted

Caesar cipher over the 31-letter Romanian alphabet.

The key conclusions drawn from this work are:

1. **Key Space vs. Practical Security:** The standard Caesar cipher features a trivial key space ($|\mathcal{K}| = 30$), making it vulnerable to instantaneous brute-force recovery. In contrast, introducing a keyword permutation expands the theoretical permutation space to $31! \approx 8.22 \times 10^{33}$, effectively eliminating brute-force attacks. However, because the cipher remains *monoalphabetic*, it preserves the statistical letter frequencies of natural Romanian language. A cryptanalyst armed with frequency tables can readily decrypt the message without knowledge of the keys.
2. **Proper Character Encoding and Arithmetic:** Handling language-specific alphabets with diacritics (such as Ă, Â, Î, Ș, Ț) requires deliberate architectural separation between internal numeric indices and platform-specific encodings (UTF-8). In accordance with the assignment specifications, computing shifts without relying on ASCII/Unicode numeric values ensures strict mathematical fidelity to Table 1.
3. **Robustness and Defensive Programming:** Accounting for keyboard variations (normalizing cedilla variants Ș/Ț to comma-below Ș/Ț) and implementing non-terminating validation loops creates a resilient, user-friendly system that prevents crashes when supplied with malformed input.

6 Source Code

The complete source code for this laboratory work is version-controlled and publicly accessible on GitHub:

[https://github.com/Vitalieww/CryptographyAndSecurityCourse/
tree/lab_1/lab_1/RomanianCaesarCipher](https://github.com/Vitalieww/CryptographyAndSecurityCourse/tree/lab_1/lab_1/RomanianCaesarCipher)

6.1 Building and Running the Project

To build and execute the application locally:

1. Clone the repository and switch to the `lab_1` branch:

```
git clone
  https://github.com/Vitalieww/CryptographyAndSecurityCourse.git
cd CryptographyAndSecurityCourse
git checkout lab_1
```

2. Navigate to the project directory:

```
cd lab_1/RomanianCaesarCipher
```

3. Build and execute with the .NET SDK:

```
dotnet run
```

References

- [1] Laboratory Work No. 1 Guide: *The Caesar Cipher and Caesar Cipher with Permutation for the Romanian Alphabet*. Technical University of Moldova, 2026.
- [2] William Stallings, *Cryptography and Network Security: Principles and Practice*, 8th Edition, Pearson, 2020.
- [3] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.